ASCIA

SMART HOME SYSTEMS

ASCIA OS

COMPLETE TECHNICAL ARCHITECTURE

System Architecture — Infrastructure — Data Flows — Security — DevOps

---

Property	Value
Document Reference	ARCH-2026-ASCIA-OS-001
Version	1.0 — February 2026
Classification	Confidential — Agency Distribution After NDA
Author	ASCIA Technical Team
Base Platform	Home Assistant OS (Apache 2.0 Fork)
Target Hardware	x86-64 Mini-PC / Tower / 1U Rack
Companion Documents	CDC ASCIA OS v1.0, CDC ASCIA Mobile App v1.0, RFP Agency v1.0

Table of Contents

 Page

Section 1 — Executive Architecture Overview

1.1 What Is ASCIA OS

ASCIA OS is a proprietary smart home operating system designed for premium residential automation. It is built as a value-added layer on top of Home Assistant OS (HAOS), leveraging HA Core's 3,000+ device integrations while replacing its entire user experience, network management, device onboarding, surveillance, AI, and diagnostic capabilities with custom ASCIA components.

The architecture follows a strict three-layer separation that allows ASCIA to benefit from upstream HA Core updates while maintaining full control over the user-facing experience and proprietary features.

1.2 Architecture Design Principles

- Zero coupling with Lovelace: the HA default frontend is entirely replaced. No end-user ever sees Home Assistant branding or UI.
- Add-on isolation: every ASCIA feature runs as an independent Docker container managed by the HA Supervisor. Failures in one add-on do not cascade to others.
- Local-first processing: all AI inference (voice, LLM, vision), all camera recording, all automation execution, and all data storage happen on the local server. No cloud dependency for core functionality.
- API-first design: every add-on exposes a documented REST or WebSocket API. The frontend is a pure API consumer with no direct database access.
- Security by default: IoT network isolation, encrypted communications, and automated key rotation require zero user intervention.
- HA Core compatibility: ASCIA OS tracks HA Core releases and maintains compatibility with its WebSocket and REST APIs. This ensures access to all 3,000+ integrations.

1.3 System Context Diagram

The following describes the complete ASCIA ecosystem and its external touchpoints:

External Entity	Connection	Protocol	Direction
IoT Devices (Zigbee)	Zigbee Coordinator (SONOFF/Conbee)	Zigbee 3.0 via Zigbee2MQTT	Bidirectional
IoT Devices (Z-Wave)	Z-Wave Controller (Zooz/Aeotec)	Z-Wave via Z-Wave JS	Bidirectional
IoT Devices (Wi-Fi)	ASCIA IoT VLAN Network	MQTT / mDNS / SSDP / UPnP	Bidirectional
IoT Devices (KNX)	KNX TP Bus Interface	KNX TP via xknx library	Bidirectional
PoE Cameras (ONVIF)	ASCIA PoE Switch	ONVIF discovery + RTSP streams	Inbound (video) + Outbound (PTZ)
PoE Network Equipment	Omada OC200 / UniFi Controller	Omada API / UniFi API	Bidirectional
User Browser (Local)	ASCIA Server Port 443	HTTPS + WebSocket (TLS 1.3)	Bidirectional
User Browser (Remote)	WireGuard VPN Tunnel	HTTPS + WebSocket over VPN	Bidirectional
ASCIA Mobile App	Local mDNS or WireGuard VPN	HTTPS + WebSocket + HLS + WebRTC	Bidirectional
ASCIA Cloud (Optional)	Cloud Relay Servers	HTTPS (relay for remote access)	Bidirectional
Encrypted Cloud Backup	ASCIA-owned Cloud Storage	HTTPS + AES-256 Encrypted Payloads	Outbound only

Internet Services	ASCIA Server WAN Port	HTTPS (Waze API, Weather, Calendar)	Outbound only
ASCIA Support (Remote)	Temporary WireGuard Session	SSH + HTTPS (time-limited, client-approved)	Inbound (controlled)

Section 2 — Three-Layer Architecture

2.1 Layer Overview

ASCIA OS is structured as three superimposed layers, each with distinct responsibilities, ownership boundaries, and update cycles. This separation is the fundamental architectural decision of the entire project.

Layer	Technology	Role	Owned by ASCIA	Update Source
Layer 1 — Foundation	HAOS (Linux + Buildroot + HA Supervisor + HA Core)	Operating system, container orchestration, state machine, 3000+ integrations, WebSocket/REST APIs	No — Unmodified upstream	HA releases (monthly)
Layer 2 — Add-ons	~12 Docker containers (Python, Node.js, Go, C)	ASCIA proprietary features: network isolation, device onboarding, NVR, AI, diagnostics, etc.	Yes — Full ASCIA development	ASCIA private repo
Layer 3 — Frontend	SvelteKit + TypeScript + Three.js	Complete branded UI replacing Lovelace: 3D dashboard, cameras, automations, settings	Yes — Full ASCIA development	ASCIA private repo

2.2 Layer 1 — HAOS Foundation (Unmodified)

2.2.1 What HAOS Provides

Home Assistant OS is a minimal Linux distribution built with Buildroot, designed specifically to run Home Assistant and its add-on ecosystem. ASCIA uses HAOS without modification to benefit from upstream security patches, driver updates, and new integrations.

Component	Technology	Function in ASCIA
Linux Kernel	Custom Buildroot kernel	Hardware abstraction, device drivers, network stack, nftables firewall
HA Supervisor	Python + Docker orchestration	Container lifecycle management, add-on installation/updates, health monitoring, snapshot/restore
HA Core	Python 3.11+ (async)	Entity state machine, automation engine, integration framework, WebSocket API, REST API
HA Database	SQLite (default)	Entity state history, event log, statistics, recorder
HA Authentication	Python (built-in)	User accounts, tokens, session management, auth providers
Container Runtime	Docker CE	Isolation for all add-ons and ASCIA components
Network Stack	NetworkManager + systemd-networkd	Base network configuration, DHCP client, DNS resolution

2.2.2 HA Core APIs Used by ASCIA

ASCIA components interact with HA Core exclusively through its published APIs. No direct access to HA Core's internal Python modules or database is permitted.

API	Protocol	Authentication	Primary ASCIA Consumer	Purpose
HA WebSocket API	WebSocket (wss://)	Long-lived access token (Bearer)	ASCIA Frontend (Layer 3)	Real-time entity state subscriptions, command execution, automation triggers
HA REST API	HTTPS	Long-lived access token (Bearer)	ASCIA Add-ons (Layer 2)	One-off operations: entity state queries, service calls, configuration reads
HA Supervisor API	HTTPS (internal)	Supervisor token (auto-injected)	ASCIA Add-ons (Layer 2)	Add-on management, system info, snapshot creation, network configuration
HA Event Bus	WebSocket subscription	Long-lived access token	ASCIA Add-ons (Layer 2)	Event-driven triggers: state changes, device registration, automation execution

2.2.3 HA Core Update Strategy

HA Core releases major updates every 4 weeks. ASCIA must maintain a disciplined update strategy to balance access to new integrations with system stability:

- Version Freeze Policy: ASCIA OS pins to a specific HA Core version. Updates are applied only after internal regression testing (minimum 7-day validation window).
- Automated Regression Testing: a CI/CD pipeline runs the full ASCIA integration test suite against each new HA Core release before approval.
- Breaking Change Detection: HA Core deprecation notices and breaking change logs are monitored. Any change affecting ASCIA APIs triggers a mandatory review.
- Rollback Capability: the ASCIA backup system can restore the previous HA Core version within 10 minutes if an update causes issues.
- Communication to Clients: HA Core updates are bundled into ASCIA OS releases with human-readable release notes. Clients never see raw HA version numbers.

2.3 Layer 2 — ASCIA Add-ons (Docker Containers)

2.3.1 Add-on Architecture Pattern

Every ASCIA add-on follows a standardized architecture pattern to ensure consistency, testability, and maintainability across the entire ecosystem:

Pattern Element	Standard	Rationale
Container Base Image	Alpine Linux 3.18+ or Debian Slim (when Alpine lacks dependencies)	Minimal attack surface, small image size (< 200MB target)
Configuration	YAML config file + environment variables injected by Supervisor	Standard HA add-on configuration mechanism
Internal API	FastAPI (Python) or Express (Node.js) on a dedicated port	Standardized REST/WebSocket interface for inter-add-on and frontend communication
Logging	Structured JSON logs to stdout (captured by Supervisor)	Centralized log collection, queryable by ascia-diagnostics
Health Check	HTTP /health endpoint returning JSON status	Supervisor health monitoring, self-healing trigger
Persistence	Mounted /data volume for persistent storage	Data survives container restarts and updates
Secret Management	Secrets injected via Supervisor, never hardcoded	Security compliance, automated rotation
Testing	Unit tests (≥ 70% coverage) + integration tests	CI/CD gate, code quality enforcement
Documentation	README.md + API schema (OpenAPI 3.0)	Developer onboarding, contract-first API design

2.3.2 Complete Add-on Registry

The following table lists every ASCIA add-on, its technical foundation, function, resource requirements, and development phase:

Add-on	Technical Base	Primary Function	Resources	Phase
ascia-network-manager	Python + NetworkManager + nftables	Automatic VLAN creation, IoT network isolation, firewall rules, DHCP/DNS for IoT subnet	Lightweight CPU	Phase 1 (MVP)
ascia-device-manager	Python + HA API	Universal device auto-discovery (PoE, Zigbee, Wi-Fi, ONVIF, KNX), onboarding wizard, automatic naming	Lightweight CPU	Phase 1 (MVP)
ascia-3d-engine	Node.js + WebSocket API	Server-side 3D scene state management, entity-to-3D mapping, GLB model management	Average CPU	Phase 1 (MVP)
ascia-nvr	Frigate + Go2RTC	NVR with GPU object detection, continuous/event recording, HLS streaming, ONVIF/RTSP management	GPU recommended	Phase 1 (MVP)
ascia-ai	Python + Ollama + Whisper + Piper	Local LLM chatbot, voice transcription, text-to-speech, automation from natural language	GPU recommended	Phase 2
ascia-automation-ai	Python + LLM API	Intelligent automation creation via natural language, pattern detection, proactive suggestions	Depends on ascia-ai	Phase 2
ascia-audio	Music Assistant	Multiroom audio management, Spotify/Deezer/Sonos integration, zone-based playback	Lightweight CPU	Phase 2

ascia-presence	Python + Zigbee2MQTT + BLE + UWB	mmWave sensor management, BLE beacon tracking, UWB precision location, room-level presence	Lightweight CPU	Phase 2
ascia-analytics	TimescaleDB + Grafana	Advanced historical data, energy monitoring, reporting, time-series analysis	Average CPU/RAM	Phase 2
ascia-vpn	WireGuard	Secure remote access, per-user VPN profiles, temporary support sessions	Lightweight CPU	Phase 1 (MVP)
ascia-diagnostics	Python + Prometheus	System monitoring, self-healing automation, remote telediagnostics, health scoring	Lightweight CPU	Phase 1 (MVP)
ascia-backup	Rclone + Custom Python	Automated local + cloud backup, encrypted off-site replication, one-click restore	Lightweight CPU	Phase 1 (MVP)

2.3.3 Inter-Add-on Communication

ASCIA add-ons communicate with each other and with HA Core through strictly defined channels. No add-on may directly access another add-on's internal database or file system.

Communication Pattern	Protocol	Use Case Example
Add-on → HA Core	HA REST API / WebSocket API (authenticated)	ascia-device-manager calls HA service to register a new entity
Add-on → Add-on	Internal REST API on Docker bridge network	ascia-automation-ai calls ascia-ai's LLM API to generate an automation
Add-on → Frontend	WebSocket (ascia-3d-engine publishes state updates)	3D engine pushes entity state changes to all connected frontends
Add-on → Devices	MQTT (Mosquitto broker) / Protocol-specific APIs	ascia-presence subscribes to Zigbee2MQTT topics for mmWave sensor data
Add-on → External	HTTPS outbound (weather, Waze, cloud backup)	ascia-analytics fetches weather data for energy correlation
Frontend → Add-on	REST API calls to add-on endpoints	Frontend requests camera snapshot from ascia-nvr

2.4 Layer 3 — ASCIA Frontend (SvelteKit)

2.4.1 Frontend Architecture

The ASCIA frontend is a SvelteKit Single Page Application that completely replaces the Home Assistant Lovelace dashboard. It serves as the sole user interface for all ASCIA OS functionality, communicating with HA Core and ASCIA add-ons exclusively through their APIs.

Component	Technology	Responsibility
Framework	SvelteKit + TypeScript	Routing, SSR, API proxy, build pipeline
3D Engine	Three.js + WebGL	3D floor plan rendering, real-time device visualization, interactive controls
State Management	Svelte stores + WebSocket subscriptions	Entity states, UI state, user session, 3D scene state
Real-time Communication	HA WebSocket API + ASCIA WebSocket APIs	Bidirectional entity state streaming, command execution
API Client	Fetch API with typed wrappers	REST calls to HA Core and ASCIA add-on endpoints
Build Output	Static SPA served by Ingress (HA Supervisor)	Single-page app accessible at ASCIA server IP on port 443
Design System	ASCIA Design System (tokens, components)	Consistent branding, dark/light themes, responsive layout
Video Streaming	HLS.js + native <video>	Camera live streams and recorded footage playback

2.4.2 Frontend-to-Backend Communication Map

The frontend never accesses databases directly. All data flows through documented APIs:

Frontend Feature	Backend API	Protocol	Data Flow
3D Dashboard (entity states)	HA WebSocket API	WebSocket	Subscribe to state_changed events → update 3D objects in real-time
3D Dashboard (3D model)	ascia-3d-engine API	REST + WebSocket	Load GLB model via REST, receive scene updates via WebSocket
Device Control (toggle, adjust)	HA WebSocket API	WebSocket	Send service call → receive state confirmation
Camera Live View	ascia-nvr API	HLS over HTTPS	Frontend requests HLS manifest → plays adaptive stream
Camera Timeline / Clips	ascia-nvr API	REST	Query recorded events, download MP4 clips
AI Chatbot	ascia-ai API	REST + WebSocket	Send natural language query → receive structured response + actions
Automation Editor	HA REST API + ascia-automation-ai	REST	Read/write automations via HA, NLP generation via ASCIA AI
Network Management	ascia-network-manager API	REST	Display VLAN status, connected devices, firewall rules
Diagnostics Dashboard	ascia-diagnostics API	REST + WebSocket	Real-time system health metrics, self-healing event log
Backup Management	ascia-backup API	REST	Trigger backup, view history, initiate restore
Energy Dashboard	ascia-analytics API	REST	Historical energy data, reports, cost

			estimates
Protocol Interfaces	Zigbee2MQTT API + HA API + Omada/UniFi API	REST + WebSocket	Zigbee mesh map, KNX addresses, PoE port status

Section 3 — Network Architecture

3.1 Network Topology

ASCIA OS creates and manages a segmented network architecture that isolates IoT devices from the home network. This is automatic and requires zero manual configuration from the customer or installer.

Network Segment	VLAN ID	Subnet	Purpose	Managed By
Home Network (Client)	Untagged / VLAN 1	Client's existing subnet	Client's personal devices (laptops, phones, TVs)	Client's modem/router
ASCIA Management	VLAN 10	10.10.10.0/24	ASCIA server, controller communication, management traffic	ascia-network-manager
IoT Devices	VLAN 100	10.100.0.0/16	All IoT devices: sensors, cameras, switches, relays, hubs	ascia-network-manager
Guest IoT (Optional)	VLAN 200	10.200.0.0/24	Guest-accessible devices (guest Wi-Fi speakers, etc.)	ascia-network-manager

3.2 Automatic Network Configuration (ascia-network-manager)

On first boot, ascia-network-manager performs the following automatic configuration sequence without any user input:

1. Detects the client's existing modem/router and determines the home network subnet.
2. Creates a dedicated IoT VLAN (VLAN 100) isolated from the home network.
3. Creates a separate Wi-Fi SSID (ASCIA-IoT) with an auto-generated unique WPA3 password.

4. Configures nftables firewall rules: IoT devices CANNOT initiate connections to the home network. The ASCIA server CAN communicate bidirectionally with both networks.
5. Deploys an internal ASCIA DNS server for IoT device name resolution.
6. Configures DHCP with reserved IP ranges per device type (cameras: 10.100.1.x, sensors: 10.100.2.x, etc.).
7. Activates basic network intrusion detection (IDS) rules monitoring for anomalous IoT behavior.

3.3 Firewall Rules (nftables)

The firewall enforces strict network segmentation:

Source	Destination	Policy	Rationale
IoT VLAN → Home Network	VLAN 100 → VLAN 1	DENY ALL	IoT devices cannot access client's personal devices
Home Network → IoT VLAN	VLAN 1 → VLAN 100	DENY ALL (except ASCIA server)	Home devices cannot directly access IoT devices
IoT VLAN → Internet	VLAN 100 → WAN	ALLOW (restricted ports)	IoT devices can reach cloud services (firmware updates, Spotify Connect)
ASCIA Server → IoT VLAN	Server → VLAN 100	ALLOW ALL	ASCIA must control all IoT devices
ASCIA Server → Home Network	Server → VLAN 1	ALLOW (port 443, 8123)	Users access ASCIA dashboard from home network
WireGuard VPN → ASCIA Server	VPN tunnel → Server	ALLOW ALL	Remote users access ASCIA via VPN
Internet → ASCIA Server	WAN → Server	DENY ALL (no port forwarding)	No direct internet exposure — VPN or ASCIA Cloud relay only

3.4 Remote Access Architecture

ASCIA OS supports two remote access methods, both of which avoid exposing any port to the public internet:

Method	Technology	Latency	Configuration Effort	Privacy Level
WireGuard VPN (Default)	WireGuard kernel module, managed by ascia-vpn add-on	< 10ms additional over base connection	Zero — user taps 'Enable VPN' in app, profile auto-installed	Maximum — all traffic stays between user and server
ASCIA Cloud Relay (Optional)	ASCIA-operated relay servers, encrypted tunnel	20-50ms additional	Zero — user enables ASCIA Cloud in settings	High — relay servers see encrypted payloads, not content

WireGuard VPN profiles are generated per-user with individual private keys. Profiles can be revoked in under 5 minutes from the ASCIA OS admin interface. Temporary support sessions (15/30/60 minutes) generate time-limited VPN profiles for ASCIA technicians.

Section 4 — Data Architecture

4.1 Database Systems

ASCIA OS uses multiple database systems, each optimized for its specific data type and access pattern:

Database	Engine	Location	Managed By	Data Stored	Retention
HA State Database	SQLite	HA Core data volume	HA Core (Recorder)	Entity state history, event log, statistics	Configurable (default 10 days)
ASCIA Analytics DB	TimescaleDB (PostgreSQL)	ascia-analytics data volume	ascia-analytics	Long-term energy data, time-series metrics, aggregated reports	1 month to 1 year (configurable)
NVR Recording DB	SQLite + filesystem	ascia-nvr data volume	ascia-nvr (Frigate)	Camera event metadata, recording index, object detection results	1 month to 1 year (configurable)
NVR Video Storage	Filesystem (MP4/HLS segments)	Data HDD/SSD (/data/nvr/)	ascia-nvr	Continuous and event-driven video recordings	Configurable, storage-aware auto-archival
Diagnostics DB	Prometheus (TSDB)	ascia-diagnostics data volume	ascia-diagnostics	System metrics: CPU, RAM, disk, network, add-on health, latency	30 days rolling
AI Knowledge DB	SQLite + vector embeddings	ascia-ai data volume	ascia-ai	Conversation history, home context, learned patterns	Indefinite (local)
Audit Log	Encrypted SQLite	ascia-security data volume	ASCIA Auth Layer	Login attempts, permission changes, API access log, key rotation events	1 year minimum
Backup Metadata	SQLite	ascia-backup data volume	ascia-backup	Backup history, restore points, cloud sync status	Indefinite

3D Scene State	In-memory + JSON persistence	ascia-3d-engine data volume	ascia-3d-engine	3D model references, entity-to-object mappings, zone definitions	Indefinite

4.2 Data Flow Diagram

The following describes the primary data flows within the ASCIA OS ecosystem:

4.2.1 Real-Time Device State Flow

- Physical device changes state (e.g., door sensor opens).
- Protocol handler (Zigbee2MQTT, MQTT broker, ONVIF) detects the change.
- HA Core receives the state update via its integration framework.
- HA Core updates the entity state in its state machine and SQLite database.
- HA Core broadcasts a state_changed event on its WebSocket API.
- ASCIA Frontend (WebSocket subscriber) receives the event and updates the 3D scene in < 100ms.
- ascia-3d-engine receives the event and updates its internal scene state for new client connections.
- ascia-analytics stores the state change in TimescaleDB for historical analysis.
- ascia-diagnostics monitors the state change for anomaly detection (self-healing triggers).

4.2.2 Camera Video Flow

- Camera streams RTSP video continuously to ascia-nvr (Frigate + Go2RTC).
- Go2RTC re-encodes the stream to HLS format for browser consumption.
- Frigate runs GPU-accelerated object detection (YOLO) on the stream in real-time.
- Detection events are stored in the NVR database with metadata (object type, confidence, bounding box).
- Video segments are written to the data storage drive (continuous or event-driven, configurable).
- Frontend requests HLS manifest from ascia-nvr API and plays adaptive stream via HLS.js.
- When an object is detected, ascia-nvr sends a notification event to HA Core, which triggers push notifications.

4.2.3 AI Voice Command Flow

- User speaks a command via microphone (ASCIA speaker, phone, or web interface).
- Audio is captured and sent to ascia-ai's Whisper endpoint for local speech-to-text transcription.
- Transcribed text is sent to the local Ollama LLM with home context (entity states, zones, user profile).
- LLM generates a structured response containing both a natural language reply and an action (HA service call).
- ascia-ai executes the action via HA REST API (e.g., turn on lights, adjust thermostat).
- Text response is sent to Piper TTS for local text-to-speech synthesis.

- Audio response is played on the appropriate speaker(s).
- Entire pipeline target: < 2 seconds without GPU, < 500ms with dedicated GPU.

4.3 MQTT Message Bus Architecture

The Mosquitto MQTT broker serves as the primary real-time message bus for IoT device communication within ASCIA OS. All MQTT traffic stays local — no messages are forwarded to external brokers.

Topic Pattern	Publisher	Subscriber(s)	Content
homeassistant/#	Zigbee2MQTT, MQTT devices	HA Core (MQTT integration)	Device discovery and state updates (HA MQTT auto-discovery)
zigbee2mqtt/#	Zigbee2MQTT	HA Core, ascia-device-manager	Zigbee device states, network events, OTA updates
frigate/#	Frigate (ascia-nvr)	HA Core, ascia-diagnostics	Camera detection events, recording notifications
ascia/presence/#	ascia-presence	HA Core, ascia-3d-engine	Room-level presence data from mmWave/BLE/UWB sensors
ascia/network/#	ascia-network-manager	ascia-diagnostics, ASCIA frontend	Network events: new device connected, firewall rule applied, intrusion detected
ascia/diagnostics/#	ascia-diagnostics	ASCIA frontend	Health metrics, self-healing events, alert notifications

Section 5 — Security Architecture

5.1 Security Layers

Security in ASCIA OS is implemented at every layer of the stack, following a defense-in-depth strategy:

Security Layer	Implementation	Standard
Network Isolation	802.1Q VLANs, nftables firewall, automatic IoT segmentation	Exceeds residential IoT best practices (NIST IR 8259)
Transport Encryption	TLS 1.3 for all HTTP/WebSocket communication	Let's Encrypt certificates + ASCIA PKI for internal communication
Authentication	HA Auth + ASCIA Auth Layer with optional 2FA (TOTP)	Token-based (Bearer tokens, long-lived access tokens)
Data Encryption at Rest	AES-256 for backups, encrypted SQLite for audit logs	ASCIA-managed encryption keys, 90-day automatic rotation
Key Management	ascia-key-manager (part of ASCIA security layer)	Automatic 90-day rotation, secure storage in /data volume
IoT Device Security	Automatic firewall rules per device, network intrusion detection	Device cannot reach home network or internet without explicit rules
Remote Access Security	WireGuard VPN with per-user keys, no port forwarding	Zero open ports to internet, revocable access in < 5 minutes
Audit Trail	Encrypted audit log with 1-year retention minimum	All login attempts, permission changes, API access recorded
Data Compliance	Privacy by design, local-first processing	GDPR + Quebec Bill 25 compliant
Penetration Testing	Annual external pen test + before each major release	Third-party security firm, results shared with ASCIA team

5.2 Authentication Architecture

ASCIA implements a dual authentication system that layers ASCIA-specific security on top of the existing HA authentication framework:

Component	Technology	Scope
HA Core Auth	Built-in HA authentication providers	User accounts, credential storage, session tokens, password policies
ASCIA Auth Layer	Custom middleware on ASCIA Frontend/API	Biometric login (via mobile app), 2FA enforcement, role-based access (admin, family, child, guest, technician), session management
API Authentication	Long-lived access tokens (Bearer scheme)	All API calls from frontend and mobile app carry an authenticated token
Add-on Authentication	Supervisor-injected tokens (automatic)	Inter-add-on communication uses Supervisor-managed service tokens
MQTT Authentication	Username/password per device	Each IoT device receives unique MQTT credentials during onboarding

5.3 User Roles & Permissions

Role	Dashboard Access	Device Control	Camera Access	Admin Settings	Automation Edit
Administrator	Full	Full	Full	Full	Full
Family Member	Full	Full (assigned zones)	Full	None	Own automations only
Child	Limited (safe devices)	Limited (lights, audio)	None	None	None
Guest	Assigned zones only	Assigned devices only	None	None	None
Service Provider	Assigned zones only	Assigned devices only	Assigned cameras	None	None
ASCIA Technician	Full (read)	Full (time-limited)	Full (time-limited)	System diagnostics only	None

Section 6 — Hardware Reference Architecture

6.1 ASCIA Server Configurations

Component	Minimum	Recommended	Premium
Processor	Intel Core i5 (8th gen+) / AMD Ryzen 5	Intel Core i7 (12th gen) / AMD Ryzen 7	Intel Core i9 / AMD Ryzen 9
RAM	16 GB DDR4	32 GB DDR4	64 GB DDR4
System Storage	256 GB NVMe SSD	512 GB NVMe SSD	1 TB NVMe SSD
Data Storage	1 TB HDD/SSD	4 TB SSD	8+ TB RAID
GPU (AI/Frigate)	None (CPU only)	NVIDIA RTX 3060 / AMD RX 6600	NVIDIA RTX 4080+
Network	1 Gbps Ethernet	2.5 Gbps Ethernet	10 Gbps Ethernet
Power Supply	65W Adapter	500W ATX	750W Gold ATX
Form Factor	Mini-PC (Intel NUC-like)	Mini-ITX Tower	ATX Tower / 1U Rack

6.2 Network Equipment

Equipment	Recommended Model(s)	Function in ASCIA	Managed By
PoE Switch	TP-Link TL-SG1008MP (8-port) or TL-SG1016PE (16-port)	Powers and connects all PoE cameras, sensors, and access points	ascia-network-manager (via Omada API)
Network Controller	TP-Link Omada OC200 or UniFi Cloud Key	Centralized management of VLANs, SSIDs, firewall rules	ascia-network-manager
Router	TP-Link ER605 or UniFi Dream Machine	WAN connection, inter-VLAN routing	ascia-network-manager
Wi-Fi Access Point	TP-Link EAP245 or UniFi 6 Lite	Broadcasts ASCIA-IoT SSID on IoT VLAN	ascia-network-manager
Zigbee Coordinator	SONOFF Zigbee 3.0 USB Dongle Plus (CC2652P)	Zigbee mesh coordinator for all Zigbee devices	Zigbee2MQTT
Z-Wave Controller	Zooz ZST39 LR or Aeotec Z-Stick 7	Z-Wave mesh controller	Z-Wave JS
UPS	APC Back-UPS 600VA or equivalent	Power protection for server and network equipment	Monitored by ascia-diagnostics

6.3 Supported IoT Protocols & Hardware

Protocol	Coordinator/Interface	Device Count	Range	ASCIA Integration
Zigbee 3.0	USB coordinator (SONOFF CC2652P)	250+ devices per coordinator	10-30m direct, mesh extends	Zigbee2MQTT add-on, 3000+ supported devices
Z-Wave	USB controller (Zooz ZST39)	232 devices per controller	30-100m direct, mesh extends	Z-Wave JS integration via HA Core
Wi-Fi (MQTT)	ASCIA IoT VLAN via Wi-Fi AP	Unlimited (network-bound)	Wi-Fi AP range	HA MQTT integration, auto-discovery
ONVIF/RTSP	PoE switch + Ethernet	Unlimited (port-bound)	100m Ethernet	ascia-nvr (Frigate + Go2RTC)
KNX TP	KNX USB/IP interface	Unlimited (bus topology)	1000m per line	xknx library via HA Core integration
BLE	Bluetooth adapter on ASCIA server + BLE proxies	Depends on proxy count	10-30m per proxy	ascia-presence add-on
UWB	UWB anchors (Samsung/Apple compatible)	4-8 anchors per installation	10-50m per anchor	ascia-presence add-on (Phase 2)
mmWave	Aqara FP2 / ASCIA mmWave sensors	1 per room (recommended)	6-8m detection range	ascia-presence add-on via Zigbee2MQTT
Matter/Thread	Thread border router (via HA Core)	Limited by Thread network	Thread mesh range	HA Core Matter integration (Phase 3-4)

Section 7 — CI/CD Pipeline & DevOps

7.1 Repository Structure

All ASCIA OS source code is hosted in a private GitHub Organization owned by ASCIA Inc. The agency has collaborator access only — never admin or owner.

Repository	Content	Branch Strategy
ascia-os	HAOS fork configuration, disk image build pipeline, OS-level customizations	main (release), develop (integration), feature/* (per-feature)
ascia-frontend	SvelteKit application (Layer 3): dashboard, 3D engine client, all UI pages	main, develop, feature/*, release/*
ascia-addons	Monorepo for all ASCIA add-ons (each add-on in a subdirectory with its own Dockerfile)	main, develop, feature/addon-name/*
ascia-mobile	Flutter application (iOS + Android + Watch): mobile app + native plugins	main, develop, feature/*, release/*
ascia-infra	Infrastructure as Code (Terraform/Ansible): cloud dev environment, CI runners, staging	main, develop
ascia-docs	Documentation: API specs (OpenAPI), architecture diagrams, runbooks, onboarding guides	main

7.2 CI/CD Pipeline

Every code push triggers an automated pipeline. No code reaches the main branch without passing all quality gates.

7.2.1 Add-on Build Pipeline (ascia-addons)

- Trigger: push to any branch, pull request to develop or main.
- Step 1 — Lint: Python (ruff/flake8), Node.js (eslint), Dart (dart analyze). Zero lint errors required.
- Step 2 — Unit Tests: pytest (Python), jest (Node.js). Minimum 70% code coverage enforced.
- Step 3 — Docker Build: each add-on builds its Docker image. Multi-arch builds (amd64 + arm64).
- Step 4 — Integration Tests: Docker Compose spins up the add-on + mock HA Core + Mosquitto. Key API flows are tested end-to-end.
- Step 5 — Security Scan: container image scanned with Trivy for known vulnerabilities.
- Step 6 — Push to Registry: on merge to main, Docker image is pushed to ASCIA private Docker Hub repository with semantic version tag.

7.2.2 Frontend Build Pipeline (ascia-frontend)

- Trigger: push to any branch, pull request to develop or main.
- Step 1 — Lint + Type Check: eslint + TypeScript strict mode. Zero errors required.
- Step 2 — Unit Tests: vitest. Minimum 60% coverage on business logic.
- Step 3 — Build: SvelteKit production build. Bundle size monitored (alert if > 2MB gzipped).
- Step 4 — Lighthouse: automated Lighthouse audit. Performance score ≥ 80, Accessibility score ≥ 90.
- Step 5 — Deploy to Staging: automatically deployed to the staging ASCIA server for manual testing.

7.2.3 Mobile Build Pipeline (ascia-mobile)

- Trigger: push to any branch, tag (v*.*.*) for releases.
- Step 1 — Lint: dart analyze. Zero errors or warnings.
- Step 2 — Unit + Widget Tests: flutter test. Minimum 70% coverage.
- Step 3 — Build iOS: via Fastlane, signed with ASCIA distribution certificate.
- Step 4 — Build Android: via Fastlane, signed with ASCIA release keystore.
- Step 5 — Distribute: TestFlight (iOS) and Google Play Internal Testing (Android) for beta testers.

7.3 Environment Strategy

Environment	Purpose	Hardware	Access
Development	Individual developer work	Cloud VM (8 vCPU, 32GB RAM)	Developer SSH via WireGuard bastion
Lab IoT (Montreal)	Physical device testing with real hardware	Mini-PC + PoE switch + IoT devices	WireGuard tunnel from cloud environments
Staging	Pre-release testing, pilot deployment validation	Cloud VM (4 vCPU, 16GB RAM) + Lab IoT tunnel	ASCIA team + agency QA via WireGuard
Production	Live customer installations	ASCIA server at customer site	ASCIA technicians only (VPN, client-approved)

7.4 Quality Gates (Mandatory for Merge)

Gate	Threshold	Enforcement
Lint Errors	0	CI/CD fails on any lint error
Unit Test Coverage (Backend)	≥ 70%	CI/CD fails below threshold
Unit Test Coverage (Frontend)	≥ 60%	CI/CD fails below threshold
All Tests Pass	100%	CI/CD fails on any test failure
Docker Image Size	< 200MB per add-on	CI/CD warning above threshold
Frontend Bundle Size	< 2MB gzipped	CI/CD warning above threshold
Security Scan (Trivy)	0 critical/high vulnerabilities	CI/CD fails on critical/high CVEs
Code Review	≥ 1 approved review	GitHub branch protection rule
Commit Signing	GPG signed commits	GitHub branch protection rule

Section 8 — Performance & Reliability

8.1 Performance KPIs

KPI	Target	Measurement Condition
Local command latency	< 100ms	Click to device execution, local network, simple command
Remote command latency	< 500ms	Via WireGuard VPN, 10+ Mbps connection
3D interface framerate (desktop)	60 FPS constant	8-room plan, 100 entities, standard desktop GPU
3D interface framerate (mobile)	30-60 FPS	5-room plan, 50 entities, iPhone 12 / Pixel 6
First display after login	< 2 seconds	Warm cache, local network
ASCIA OS cold boot	< 90 seconds	From server power-on to accessible interface
PoE device discovery	< 5 seconds	ASCIA switch, new device connected
ONVIF camera discovery	< 20 seconds	Local network scan
Camera stream latency	< 500ms	HLS local, 1080p, 100 Mbps network
AI voice response	< 2s without GPU, < 500ms with GPU	Whisper small.en model, end of speech to action
Crash rate	< 0.01% per month	Per installation, no planned restarts
System availability (local)	99.5% per month	Excluding internet outages

8.2 Reliability Requirements

- ASCIA OS must operate 24/7 without human intervention for a minimum of 6 months.
- Non-critical add-on updates deploy with zero downtime (rolling container restart).
- ASCIA Core updates (HA Core + add-ons) require a reboot of less than 60 seconds.
- Backup restoration completes in less than 10 minutes for system configuration.
- Self-healing: ascia-diagnostics detects failures and automatically attempts remediation (service restart, network reconnection, configuration reload) before alerting the customer.
- If automatic remediation fails, an ASCIA support ticket is created automatically.

8.3 Self-Healing Architecture

The ascia-diagnostics add-on implements a three-tier automatic recovery system:

Tier	Trigger	Automatic Action	Notification
Tier 1 — Soft Recovery	Add-on health check fails (2 consecutive)	Restart the affected add-on container	Silent log entry (no customer notification)
Tier 2 — Service Recovery	Add-on fails to recover after Tier 1 (3 attempts)	Full container rebuild + data volume check + network reconnection	Customer notification: 'System performed maintenance, all services restored'
Tier 3 — Escalation	Tier 2 fails or hardware-level issue detected	Create ASCIA support ticket with full diagnostic bundle	Customer notification: 'Issue detected, ASCIA support has been notified'

Section 9 — Update & Deployment Strategy

9.1 ASCIA OS Image Build

ASCIA OS is distributed as a complete disk image that can be flashed onto the ASCIA server hardware. The image includes HAOS, all pre-installed ASCIA add-ons, and the ASCIA frontend.

- Build process: GitHub Actions triggers a build that forks the latest validated HAOS image, injects the ASCIA add-on repository, pre-installs all ASCIA add-ons, and packages the ASCIA frontend.
- Output: a single .img.gz file that can be written to the server's NVMe SSD using a standard imaging tool.
- First boot: the image boots, runs the ASCIA initialization wizard (network detection, user creation, device discovery), and the system is operational within 5 minutes.

9.2 Over-the-Air (OTA) Updates

After initial installation, ASCIA OS updates are delivered over-the-air through the ASCIA update channel:

Update Type	Mechanism	Downtime	Frequency	User Control
Add-on Update	Docker image pull from ASCIA registry + container restart	Zero (rolling restart)	As needed (bug fixes, features)	Manual or auto (configurable)
Frontend Update	New SvelteKit build deployed to Ingress	Zero (hot swap)	As needed	Automatic (transparent)
HA Core Update	Supervisor-managed HA Core container update	< 60 second reboot	Monthly (after ASCIA validation)	Manual (user approves)
HAOS Update	Full OS partition swap + reboot	< 90 second reboot	Quarterly	Manual (user approves)
Critical Security Patch	Emergency pipeline: add-on or OS update pushed with priority flag	Minimal (depends on scope)	As needed	Auto-install with notification

9.3 Rollback Strategy

- Every update creates an automatic snapshot before applying changes.
- If an update fails or causes issues, the system can be rolled back to the pre-update state within 10 minutes.
- HA Core rollback: the Supervisor can downgrade to the previously installed HA Core version.
- Add-on rollback: Docker images are tagged with semantic versions; reverting to a previous tag restores the previous version.
- HAOS rollback: HAOS maintains an A/B partition scheme. If the new partition fails to boot, the system automatically reverts to the previous partition.

Section 10 — API Reference Summary

10.1 ASCIA API Ecosystem

The following APIs form the complete ASCIA OS API surface. All APIs require authentication and use TLS 1.3 encryption.

API	Protocol	Authentication	Primary Consumer	Key Endpoints
HA WebSocket API	WebSocket (wss://)	Bearer Token	ASCIA Frontend, Mobile App	auth, subscribe_events, call_service, get_states
HA REST API	HTTPS	Bearer Token	ASCIA Add-ons, Frontend	/api/states, /api/services, /api/config, /api/history
ASCIA Device API	REST (HTTPS)	ASCIA Token	ascia-device-manager, Frontend	/devices, /devices/discover, /devices/{id}/pair, /devices/{id}/rename
ASCIA 3D API	WebSocket + REST	ASCIA Token	ASCIA Frontend	/scene/state, /scene/model, /scene/zones, ws://subscribe
ASCIA AI API	REST + WebSocket	ASCIA Token	ASCIA Frontend, Mobile App	/chat, /voice/transcribe, /voice/synthesize, /automations/generate
ASCIA NVR API	REST + HLS	ASCIA Token	ASCIA Frontend, Mobile App	/cameras, /cameras/{id}/stream, /cameras/{id}/events, /clips
ASCIA Analytics API	REST	ASCIA Token	ASCIA Frontend	/energy, /reports, /history, /aggregations
ASCIA Network API	REST	ASCIA Token	ASCIA Frontend	/vlans, /devices, /firewall, /dns, /dhcp
ASCIA Diagnostics API	REST + WebSocket	ASCIA Token	ASCIA Frontend, Support	/health, /metrics, /self-healing/log, /tickets
ASCIA Backup API	REST	ASCIA Token	ASCIA Frontend	/backups, /backups/create, /backups/{id}/restore, /backups/schedule

MQTT Broker	MQTT/MQTTS (1883/8883)	Username/Password	IoT Devices, Add-ons	homeassistant/#, zigbee2mqtt/#, frigate/#, ascia/#

10.2 API Versioning Strategy

- All ASCIA APIs use semantic versioning (v1, v2, etc.) in the URL path.
- Breaking changes require a major version bump and a 6-month deprecation notice.
- Each major version includes a migration guide documenting all changes.
- The frontend negotiates the API version with each add-on on connection and adapts its behavior accordingly.
- HA Core APIs are consumed as-is (ASCIA does not version-wrap HA APIs).

Section 11 — Licensing & Legal Compliance

11.1 Open-Source License Compliance

Component	License	ASCIA Obligation
Home Assistant OS	Apache 2.0	Attribution required in credits. Commercial use permitted.
Home Assistant Core	Apache 2.0	Attribution required. No modification of HA Core source.
Frigate	MIT	No restriction on commercial use.
Zigbee2MQTT	GPL v3	Used as a separate add-on (Docker container). No linking with ASCIA proprietary code. Compliant.
Music Assistant	Apache 2.0	Attribution required. Commercial use permitted.
WireGuard	GPL v2 (kernel module)	Integrated in Linux kernel. No modification. Compliant.
Mosquitto MQTT	EPL v2 + EDL	Commercial use permitted without restriction.
Ollama	MIT	No restriction on commercial use.
Whisper (OpenAI)	MIT	Local model execution. No cloud dependency.
Three.js	MIT	No restriction on commercial use.
SvelteKit	MIT	No restriction on commercial use.
Flutter	BSD 3-Clause	No restriction on commercial use.

Note: This table is for informational purposes. A full legal review by a qualified intellectual property attorney is recommended before the commercial launch of ASCIA OS.

11.2 Data Privacy Compliance

- GDPR Compliance: all personal data (camera recordings, presence tracking, voice recordings) is stored locally on the ASCIA server. No personal data is transmitted to ASCIA Inc. or any third party without explicit customer consent.
- Quebec Bill 25 Compliance: privacy by design. Data collection is minimized to what is necessary for system operation. Customers can delete all their data at any time.
- ASCIA Cloud (Optional): when enabled, the cloud relay handles only encrypted tunnel establishment. No user data transits through ASCIA Cloud servers.
- Telediagnostics (Optional): when enabled by the customer, only system health data (CPU, RAM, disk, device status, battery levels, network metrics) is transmitted to ASCIA. No camera footage, voice recordings, or personal content is included.

Section 12 — Glossary

Term	Definition
HAOS	Home Assistant Operating System — the Buildroot-based Linux distribution on which ASCIA OS is built
HA Core	Home Assistant Core — the Python backend managing entities, states, automations, and integrations
HA Supervisor	Container orchestration layer managing add-ons, snapshots, and system health
Lovelace	HA's default frontend, completely replaced by ASCIA's SvelteKit frontend
Entity	A logical representation of a device or attribute in HA (e.g., light.living_room, sensor.door_bedroom1)
Add-on	A Docker container managed by HA Supervisor providing additional functionality
Blueprint	A configurable, reusable automation template in Home Assistant
VLAN	Virtual LAN — a network segmentation technique isolating IoT devices from the home network
nftables	Linux kernel firewall framework used by ascia-network-manager for traffic filtering
MQTT	Message Queuing Telemetry Transport — lightweight IoT messaging protocol
Zigbee2MQTT	Gateway converting Zigbee devices into MQTT/HA compatible entities
ONVIF	Open Network Video Interface Forum — interoperability standard for IP cameras
RTSP	Real-Time Streaming Protocol — video streaming protocol used by IP cameras

HLS	HTTP Live Streaming — adaptive video streaming protocol for camera feeds
Frigate	Open-source NVR with GPU object detection, used as the ascia-nvr engine
Go2RTC	Real-time camera re-streaming engine providing HLS/WebRTC output from RTSP sources
WireGuard	Modern VPN protocol — fast, secure, simple — used for remote ASCIA access
Whisper	OpenAI's open-source speech-to-text model, executed locally in ascia-ai
Ollama	Local LLM server running language models (Llama3, Mistral) on the ASCIA server
Piper	Open-source text-to-speech engine for local voice synthesis
xknx	Python library for KNX TP bus communication
mmWave	Millimeter-wave radar for static and dynamic presence detection
UWB	Ultra-Wideband — radio technology for centimeter-precise indoor positioning
GLB/GLTF	Standard binary 3D file format used for ASCIA 3D models
Three.js	JavaScript library for WebGL 3D rendering in the browser
SvelteKit	Compiled JavaScript framework used for the ASCIA frontend
TimescaleDB	PostgreSQL extension for time-series data, used in ascia-analytics

Prometheus	Monitoring system with time-series database, used in ascia-diagnostics
Grafana	Data visualization platform for internal ASCIA diagnostics dashboards

---

ASCIA Smart Home Systems

ASCIA OS — Complete Technical Architecture — Version 1.0